

NiceGUI 测试中 Screen Fixture 详细解析

在 NiceGUI 自动化测试体系中，**Screen Fixture** 是基于 `pytest` 夹具（Fixture）机制封装的、用于快速初始化 `nicegui.testing.screen_plugin.Screen` 对象的复用组件。它与 User Fixture 对应，前者聚焦“全局 UI 状态验证、视觉对比”，后者聚焦“用户交互模拟”，共同构成 NiceGUI 测试的核心复用底座。Screen Fixture 封装了应用启动、`screen_plugin` 初始化、环境销毁等重复逻辑，保证测试用例的简洁性、环境的一致性与隔离性，是工程化测试中验证全局 UI 状态的必备夹具。

一、核心定位与设计初衷

使用 `screen_plugin` 编写测试用例时，需重复执行以下步骤：

1. 调用 `testing.run_app()` 启动 NiceGUI 应用并绑定 Playwright 页面；
2. 初始化 `screen_plugin.Screen` 对象；
3. 测试结束后清理应用进程与浏览器上下文。

Screen Fixture 针对这些重复操作做了封装：

- **复用性**：一次定义，所有测试用例可直接注入 `Screen` 对象，无需重复初始化；
- **隔离性**：支持灵活配置作用域（function/module/session），控制测试环境的隔离粒度；
- **统一性**：集中管理 `screen_plugin` 的初始化参数（如浏览器类型、超时配置），避免分散配置导致的不一致；
- **扩展性**：可与 User Fixture 组合，实现“交互模拟 + 全局 UI 验证”的完整测试闭环。

二、Screen Fixture 的基础定义与使用

1. 核心定义模板（`conftest.py`）

Screen Fixture 需定义在 `conftest.py` 文件中（pytest 自动识别），以下是最基础的定义：

```
# conftest.py（测试目录根目录）
import pytest
from nicegui import testing
from nicegui.testing import screen_plugin
from app import main # 导入你的 NiceGUI 应用入口函数

@pytest.fixture(scope="function") # 作用域：每个测试用例独立初始化
def screen():
    """Screen Fixture: 初始化 screen_plugin.Screen 对象，绑定运行中的 NiceGUI 应用"""
    # 启动应用并获取 Playwright Page 对象（核心：隔离测试环境）
    with testing.run_app(
        app_function=main,
        port=None, # 随机端口，避免多线程/多用例端口冲突
        browser="chromium", # 指定测试浏览器（chromium/firefox/webkit）
        headless=True, # 无头模式（CI/CD 推荐，调试时可设为 False）
        native=False, # 禁用 NiceGUI 原生窗口（测试必须）
        reload=False # 禁用热重载（避免测试中应用重启）
    ) as page:
        # 初始化 Screen 对象：绑定 Playwright 页面与 screen_plugin
        screen_obj = screen_plugin.Screen(page)
```

```
# 可选：全局配置 Screen 超时（默认 5000ms）
screen_obj.default_timeout = 3000
yield screen_obj # 将 Screen 对象注入测试用例
# with 上下文自动销毁：关闭应用、销毁浏览器上下文，无需手动清理
```

2. 测试用例中使用 Screen Fixture

```
# test_app.py
def test_global_text_verification(screen): # 直接注入 screen fixture
    # 1. 全局断言页面包含指定文本（screen_plugin 核心能力）
    screen.assert_text("用户注册")
    screen.assert_text("用户名")
    screen.assert_text("注册按钮")

    # 2. 断言页面不包含错误文本
    screen.assert_no_text("服务器异常")
    screen.assert_no_text("验证失败")

    # 3. 统计指定文本的元素数量
    assert screen.count_text("输入框") == 2 # 页面包含2个带“输入框”文本的组件

    # 4. 验证元素可见性
    assert screen.is_visible("用户注册标题")
    assert not screen.is_visible("加载中提示")

def test_dynamic_content(screen):
    # 等待异步加载的文本出现（适配后台任务/动态渲染）
    screen.wait_for_text("数据加载完成", timeout=4000)
    # 断言动态内容可见
    assert screen.is_visible("数据加载完成")
```

三、Screen Fixture 的关键配置

1. 作用域（scope）配置

Fixture 作用域决定了 `Screen` 对象与测试环境的生命周期，核心可选值：

作用域	行为	适用场景
<code>function</code> （默认）	每个测试用例启动一次应用、创建一个 <code>Screen</code> 对象	需严格隔离的用例（如修改全局 UI 状态 的用例）
<code>module</code>	整个测试模块（.py 文件）共享一个 <code>Screen</code> 对象	同模块用例无状态冲突，追求测试效率
<code>session</code>	整个测试会话（所有用例）共享一个 <code>Screen</code> 对象	大型应用（启动耗时久），需确保用例无 状态污染

示例：module 作用域的 Screen Fixture

```

@pytest.fixture(scope="module")
def screen():
    # 整个模块仅启动一次应用，所有用例复用同一个 Screen 对象
    with testing.run_app(main) as page:
        screen_obj = screen_plugin.Screen(page)
        yield screen_obj
    # 模块所有用例执行完毕后才销毁应用

```

2. 自定义 Screen 初始化参数

可在 Fixture 中配置 `screen_plugin.Screen` 的全局参数，统一管控测试行为：

```

@pytest.fixture(scope="function")
def screen():
    with testing.run_app(main, browser="firefox") as page:
        screen_obj = screen_plugin.Screen(page)
        # 全局设置超时（所有断言/等待方法默认使用该值）
        screen_obj.default_timeout = 5000
        # 配置视觉对比的默认误差阈值（默认 0.01）
        screen_obj.default_screenshot_threshold = 0.02
        yield screen_obj

```

3. 前置操作与状态重置

可在 Fixture 中添加前置操作（如等待页面加载完成、重置 UI 初始状态），确保每个用例的初始环境一致：

```

@pytest.fixture(scope="function")
def screen():
    with testing.run_app(main) as page:
        screen_obj = screen_plugin.Screen(page)
        # 前置操作：等待页面核心元素加载完成
        screen_obj.wait_for_text("首页", timeout=3000)
        # 前置操作：清空所有输入框（重置初始状态）
        input_locators = screen_obj.page.locator("input")
        for locator in input_locators.all():
            locator.clear()
        yield screen_obj

```

四、进阶用法

1. Screen Fixture 与 User Fixture 组合使用

实际测试中，常需先模拟用户交互，再验证全局 UI 状态，可封装组合夹具：

```

# conftest.py
@pytest.fixture(scope="function")
def test_env():
    """组合夹具：同时返回 user 和 screen 对象"""
    with testing.run_app(main) as page:
        user = user_plugin.Page(page)
        screen = screen_plugin.Screen(page)
        yield {"user": user, "screen": screen}

```

```
# test_app.py
def test_interact_and_verify(test_env):
    # 1. User Fixture 模拟交互
    user = test_env["user"]
    user.type(user.get_by_label("用户名"), "test_user123")
    user.click(user.get_by_text("注册"))

    # 2. Screen Fixture 全局验证
    screen = test_env["screen"]
    screen.assert_text("test_user123") # 断言输入的用户名显示在页面
    screen.wait_for_no_text("加载中") # 等待加载状态消失
    screen.assert_text("注册成功") # 断言注册成功提示
```

2. 参数化 Screen Fixture

通过 `@pytest.mark.parametrize` 为 Screen Fixture 传入参数，适配多场景测试（如不同浏览器、不同基准截图）：

```
# conftest.py
@pytest.fixture(scope="function")
def screen(request):
    # 获取参数化的浏览器类型（默认 chromium）
    browser = request.param if hasattr(request, "param") else "chromium"
    with testing.run_app(main, browser=browser) as page:
        yield screen_plugin.Screen(page)

# test_app.py
import pytest

# 参数化：测试不同浏览器下的 UI 一致性
@pytest.mark.parametrize("screen", ["chromium", "firefox", "webkit"], indirect=True)
def test_cross_browser_ui(screen):
    screen.assert_text("用户注册")
    screen.assert_text("注册按钮")

    # 验证核心组件数量一致
    assert screen.count_text("输入框") == 2
```

3. 异步版本的 Screen Fixture

针对异步测试用例（`pytest.mark.asyncio`），需定义异步版 Screen Fixture：

```
# conftest.py
import pytest
import asyncio
from nicegui import testing
from nicegui.testing import screen_plugin

@pytest.fixture(scope="function")
async def async_screen():
    """异步 Screen Fixture"""
    async with testing.async_run_app(main) as page:
```

```

        screen_obj = screen_plugin.Screen(page)
        yield screen_obj

# test_app.py
@pytest.mark.asyncio
async def test_async_visual_verify(async_screen):
    # 异步等待动态文本出现
    await async_screen.wait_for_text("异步加载完成")
    # 异步断言文本存在
    assert await async_screen.is_visible("异步加载完成")
    # 异步生成截图（视觉对比）
    await async_screen.screenshot("screenshots/async_test.png")

```

4. 视觉对比专用 Screen Fixture

针对视觉对比场景，可封装专用 Fixture，预设截图路径、误差阈值等：

```

# conftest.py
import os
from pathlib import Path

@pytest.fixture(scope="function")
def visual_screen():
    # 创建基准截图目录
    baseline_dir = Path("baseline_screenshots")
    baseline_dir.mkdir(exist_ok=True)

    with testing.run_app(main, headless=True) as page:
        screen_obj = screen_plugin.Screen(page)
        screen_obj.default_screenshot_threshold = 0.01 # 严格的误差阈值
        # 绑定基准截图路径
        screen_obj.baseline_dir = baseline_dir
        yield screen_obj

# test_app.py
def test_visual_consistency(visual_screen):
    # 生成当前截图并与基准截图对比
    visual_screen.assert_screenshot_match(
        baseline_path=visual_screen.baseline_dir / "register_form.png",
        full_page=True
    )

```

五、关键注意事项

1. 环境隔离与状态污染

- 若 Fixture 作用域为 `module/session`，需确保测试用例之间无 UI 状态污染（如避免前一个用例修改的文本影响后一个用例的断言）；
- 随机端口（`port=None`）是默认推荐配置，避免多线程 / 多用例并行时端口冲突。

2. 视觉对比的特殊配置

- 视觉对比对浏览器分辨率、渲染引擎敏感，建议在 Fixture 中固定浏览器窗口大小：

```
@pytest.fixture(scope="function")
def screen():
    with testing.run_app(main) as page:
        page.set_viewport_size({"width": 1920, "height": 1080}) # 固定窗口大小
    yield screen_plugin.Screen(page)
```

- 动态内容（如时间戳、随机数）需通过 `mask` 参数遮罩，避免视觉对比误判。

3. 调试技巧

- 调试时将 `headless=False`，可直观查看页面的全局状态；
- 结合 `pytest -s` 运行测试，输出 `Screen` 对象的日志，定位断言失败原因（如文本未找到的具体位置）。

4. CI/CD 适配

- CI 环境中需安装 Playwright 系统依赖（`playwright install-deps`）；
- 视觉对比测试建议在固定环境（如 Docker 容器）中运行，避免不同环境的渲染差异。

六、Screen Fixture 与原生 screen_plugin 的对比

特性	Screen Fixture	直接使用 screen_plugin
代码复用	一次定义，所有用例复用	每个用例需重复编写启动 / 初始化代码
配置统一性	集中配置浏览器、超时、误差阈值	分散配置，易出现不一致
环境隔离	可通过 scope 灵活控制	需手动启动新应用实现隔离
学习成本	需掌握 pytest 夹具基础	仅需掌握 screen_plugin API
适用场景	工程化测试、多用例验证	单例调试、临时验证